

Aufgabe Klausurvorbereitung – EcoTracker

Kontext

Du arbeitest an einem System für ein großes Naturreservat (EcoTracker). Ranger melden täglich ihre Tierbeobachtungen, die durch das System gefiltert, sortiert, protokolliert und für Berichte als Textdatei exportiert werden sollen. Deine Aufgabe ist es, das System auszuprogrammieren und dabei die Grundkonzepte der Java-Programmierung korrekt anzuwenden.

1) Das Datenmodell

Wir verwenden für die Beobachtungen ein record *AnimalObservation*:

- String species (Tierart, z.B. „Fuchs“)
- String area (Gebiet im Reservat, z.B. „Sektor A“)
- Int count (Anzahl der gesicherten Tiere)

2) Aufgaben für die Klasse EcoTrackerManager

Implementiere die folgenden Aufgaben in der Klasse *EcoTrackerManager*.

Erstelle zuerst in der Klasse einen Logger Mittels

```
Logger.getLogger(EcoTrackerManager.class.getName())
```

Aufgabe 1: Exceptions & Logging

Die Methode *addObservation(AnimalObservation observation)* soll eine neue Beobachtung in die *observations* Liste einfügen.

- Prüfe, ob die Anzahl (*count*) der gesichteten Tiere größer als 0 ist.
- Falls ja:
 - Logge die erfolgreiche Beobachtung mit dem *LOGGER*. Benutze das *Info*-Level.
 - Füge die Beobachtung zur Liste hinzu.
- Falls nein:
 - Logge den Fehler mit dem *LOGGER*. Benutze das *Warning*-Level.
 - Werfe eine selbstgeschriebene *InvalidObservationException* (Checked Exception).

Aufgabe 2: Comparator & Datenstrukturen

Die Ranger benötigen eine Übersicht, bei der zu jeder Beobachtung eine Notiz gespeichert wird.

- Erstelle in der Methode *getGroupedObservations()* eine *TreeMap<AnimalObservation, String>*.
- Da ein record standardmäßig nicht sortierbar ist, musst du der *TreeMap* einen **Comparator** übergeben.
- Der Comparator soll primär alphabetisch nach der Tierart (*species*) und sekundär (bei gleicher Tierart) absteigend nach der Anzahl (*count*) sortieren.
- Füge alle bisherigen Beobachtungen aus deiner Liste in die *TreeMap* ein (als Value kannst du den String "Sichtung bestätigt" nutzen) und gib die Map zurück.

Aufgabe 3: Funktionale Programmierung & Streams

Um den Code sauber zu halten, lagern wir Lambda-Ausdrücke in eigenständige Variablen aus.

- Erstelle in der Methode *generateAlertList()* ein Variable vom Typ *Predicate*, welcher prüft, ob die Tierart "Wolf" oder "Bär" ist.
- Erstelle eine weitere Variable vom Typ *Function*, die eine Beobachtung in das Format "ACHTUNG: [count] x [species] in [area] gesichtet!" umwandelt.
- Nutze nun die Java Stream API auf deiner Beobachtungs-Liste: Filtere mit dem *Predicate*, wandle die Elemente mit der *Function* um und sammle das Ergebnis in einer *List*.
- Gib diese Liste zurück.

Aufgabe 4: IO-Operationen

Die in Aufgabe 3 generierte Warnliste muss auf der Festplatte gesichert werden.

- Schreibe die Methode *saveAndReadAlerts(List<String> alerts)*.
- Schreibe die übergebene Liste Zeile für Zeile als einfachen Klartext in eine Datei namens „alerts.txt“.
- Lies die Datei danach direkt wieder aus der Datei aus
 - Gib jede Zeile mit dem *LOGGER* in der Konsole aus. Dies beweist, dass der Schreibvorgang erfolgreich war.

3) Main-Klasse zum Testen

Die Main-Klasse wurde schon programmiert, sodass du das *EcoTracker* Programm ausführen und testen kannst. Entferne dazu die Kommentare am Anfang und am Ende der Klasse.